전체 구조 (Overall Architecture)

개요

Scroll의 ZKEVM은 이더리움 가상 머신(EVM)의 실행을 영지식 증명(Zero-Knowledge Proof)으로 검증하는 시스템입니다. 이 문서는 ZKEVM 구현체의 전체 아키텍처와 각 레이어의 역할을 설명합니다.

레이어별 구조

ZKEVM 시스템은 크게 4개의 레이어로 구성됩니다:

```
┌──────────────────────────────────────────────────────────────────────┐
│ 1. Execution Layer │
│ (Geth / EVM Execution) │
└──────────────────────────────┬──────────────────────────────────────┘
│ Execution Trace
↓
┌──────────────────────────────────────────────────────────────────────┐
│ 2. Bus-Mapping Layer │
│ (Trace → Circuit Input 변환) │
└──────────────────────────────┬──────────────────────────────────────┘
│ Circuit Witnesses
↓
```

```
┌──────────────────────────────────────────────────────────────────────┐
│ 3. Circuit Layer │
│ (제약 조건 정의 및 검증) │
│ ┌──────────┬──────────┬──────────┬──────────┬──────────┐ │
│ │ EVM │ State │ Bytecode │ Keccak │ Tx │ │
│ │ Circuit │ Circuit │ Circuit │ Circuit │ Circuit │ │
│ └──────────┴──────────┴──────────┴──────────┴──────────┘ │
│ ┌──────────┬──────────┬──────────┬──────────┬──────────┐ │
│ │ Copy │ Sig │ MPT │ SHA256 │ MODEXP │ │
│ │ Circuit │ Circuit │ Circuit │ Circuit │ Circuit │ │
│ └──────────┴──────────┴──────────┴──────────┴──────────┘ │
│ └──────────┴──────────┴──────────┴──────────┴──────────┘ │
│ │ ECC │ Exp │ RLP │ Poseidon │ PI │ │
│ │ Circuit │ Circuit │ Circuit │ Circuit │ Circuit │ │
│ └──────────┴──────────┴──────────┴──────────┴──────────┘ │
└──────────────────────────────┬───────────────────────────────────────┘
│ Proof
↓
┌──────────────────────────────────────────────────────────────────────┐
│ 4. Proof Layer │
│ (증명 생성 및 집계) │
└──────────────────────────────────────────────────────────────────────┘
```

1. Execution Layer (실행 레이어)

역할: 이더리움 트랜잭션을 실제로 실행하고 실행 트레이스를 생성합니다.

주요 컴포넌트:

- Geth 트레이서: 수정된 Geth 클라이언트가 EVM 실행 과정을 추적
- `GethExecTrace` : 각 opcode 실행 단계의 상태를 기록

출력:

- 실행 트레이스 (`GethExecTrace`)
- 각 단계의 스택, 메모리, 스토리지 상태
- Gas 사용량 및 실행 결과

2. Bus-Mapping Layer (버스 매핑 레이어)

역할: 실행 트레이스를 회로가 이해할 수 있는 형태로 변환합니다.

주요 컴포넌트:

- `CircuitInputBuilder` : 트레이스를 회로 입력으로 변환하는 핵심 구조체
- `ExecStep` : 각 실행 단계를 표현
- `RwOp` : Read-Write 연산을 추상화

처리 과정:

```
// bus-mapping/src/circuit_input_builder.rs
pub struct CircuitInputBuilder {
pub sdb: StateDB, // 상태 데이터베이스
pub code_db: CodeDB, // 코드 데이터베이스
pub block: Blocks, // 블록 데이터
```

```
pub block_ctx: BlockContext, // 블록 컨텍스트
}
```

출력:

- `Block` : 회로에 필요한 모든 witness 데이터
- RW 연산 목록
- 각 회로별 입력 데이터

3. Circuit Layer (회로 레이어)

역할: 영지식 증명 제약 조건을 정의하고 witness를 검증합니다.

SuperCircuit 구조

`SuperCircuit` 은 모든 하위 회로를 통합하는 최상위 회로입니다:

```
// zkevm-circuits/src/super_circuit.rs
pub struct SuperCircuit<F: Field, ...> {
pub evm_circuit: EvmCircuit<F>,
pub state_circuit: StateCircuit<F>,
pub tx_circuit: TxCircuit<F>,
pub pi_circuit: PiCircuit<F>,
pub bytecode_circuit: BytecodeCircuit<F>,
pub copy_circuit: CopyCircuit<F>,
pub exp_circuit: ExpCircuit<F>,
pub keccak_circuit: KeccakCircuit<F>,
pub sha256_circuit: SHA256Circuit<F>,
```

```
pub poseidon_circuit: PoseidonCircuit<F>,
pub sig_circuit: SigCircuit<F>,
pub modexp_circuit: ModExpCircuit<F>,
pub ecc_circuit: EccCircuit<F, 9>,
pub rlp_circuit: RlpCircuit<F, Transaction>,
#[cfg(feature = "zktrie")]
pub mpt_circuit: MptCircuit<F>,
}
```

핵심 회로 (Core Circuits)

1. **EVM Circuit**: EVM opcode 실행 검증
2. **State Circuit**: 상태 변경 및 RW 테이블 검증
3. **Bytecode Circuit**: 스마트 컨트랙트 바이트코드 검증
4. **Keccak Circuit**: Keccak 해시 함수 검증
5. **Tx Circuit**: 트랜잭션 서명 및 RLP 인코딩 검증
6. **Copy Circuit**: 메모리/스토리지 복사 연산 검증
7. **Sig Circuit**: ECDSA 서명 검증

고급 회로 (Advanced Circuits)

8. **MPT Circuit**: Merkle Patricia Trie 증명 검증
9. **SHA256 Circuit**: SHA256 프리컴파일 검증
10. **MODEXP Circuit**: 모듈러 지수 연산 검증
11. **ECC Circuit**: 타원곡선 연산 검증

12. **RLP Circuit**: RLP 인코딩 검증
13. **Poseidon Circuit**: Poseidon 해시 검증
14. **Exp Circuit**: 지수 연산 검증
15. **PI Circuit**: Public Input 검증

회로 간 데이터 공유: 테이블 시스템

회로들은 공유 테이블을 통해 데이터를 교환합니다:

```
// zkevm-circuits/src/super_circuit.rs
pub struct SuperCircuitConfig<F: Field> {
block_table: BlockTable,
mpt_table: MptTable,
rlp_table: RlpTable,
tx_table: TxTable,
poseidon_table: PoseidonTable,
// ... 기타 테이블들
}
```

주요 테이블:

- `RwTable` : Read-Write 연산 기록
- `TxTable` : 트랜잭션 데이터
- `BytecodeTable` : 바이트코드 데이터
- `BlockTable` : 블록 메타데이터
- `CopyTable` : 복사 연산 추적

- `KeccakTable` : Keccak 해시 입출력
- `MptTable` : MPT 증명 데이터

4. Proof Layer (증명 레이어)

역할: 개별 회로의 증명을 생성하고 집계합니다.

주요 컴포넌트:

- `prover` : 개별 증명 생성
- `aggregator` : 여러 증명을 하나로 집계
- Snark-verifier: 증명 검증

각 레이어의 책임과 인터페이스

Execution → Bus-Mapping 인터페이스

입력: `GethExecTrace`

```
pub struct GethExecTrace {
pub struct_logs: Vec<GethExecStep>,
pub gas: Gas,
pub failed: bool,
pub return_value: String,
// ...
}
```

출력: `Block` (witness 데이터)

Bus-Mapping → Circuit 인터페이스

입력: `Block`

```
pub struct Block {
pub txs: Vec<Transaction>,
pub rws: RwMap,
pub bytecodes: CodeDB,
pub circuits_params: CircuitsParams,
// ...
}
```

출력: 각 회로의 witness 할당

Circuit → Proof 인터페이스

입력: 회로 인스턴스 및 witness

출력: 영지식 증명 (Proof)

SuperCircuit 설정 과정

`SuperCircuit` 의 설정은 다음 순서로 진행됩니다:

```
// zkevm-circuits/src/super_circuit.rs
impl SubCircuitConfig<Fr> for SuperCircuitConfig<Fr> {
fn new(meta: &mut ConstraintSystem<Fr>, ...) -> Self {
// 1. 테이블 생성
```

```
let tx_table = TxTable::construct(meta);
let rw_table = RwTable::construct(meta);
let bytecode_table = BytecodeTable::construct(meta);
// ... 기타 테이블들
// 2. 회로 설정 (순서 중요!)
let keccak_circuit = KeccakCircuitConfig::new(meta, ...);
let sha256_circuit = SHA256CircuitConfig::new(meta, ...);
let poseidon_circuit = PoseidonCircuitConfig::new(meta, ...);
let rlp_circuit = RlpCircuitConfig::new(meta, ...);
let pi_circuit = PiCircuitConfig::new(meta, ...);
let tx_circuit = TxCircuitConfig::new(meta, ...);
let bytecode_circuit = BytecodeCircuitConfig::new(meta, ...);
let copy_circuit = CopyCircuitConfig::new(meta, ...);
let mpt_circuit = MptCircuitConfig::new(meta, ...);
let modexp_circuit = ModExpCircuitConfig::new(meta, ...);
let state_circuit = StateCircuitConfig::new(meta, ...);
let exp_circuit = ExpCircuitConfig::new(meta, ...);
let evm_circuit = EvmCircuitConfig::new(meta, ...);
// 3. Sig와 ECC Circuit은 마지막에 설정
// (halo2-lib의 수직 할당 게이트 사용)
let sig_circuit = SigCircuitConfig::new(meta, ...);
let ecc_circuit = EccCircuitConfig::new(meta, ...);
// 4. 설정 반환
```

```
SuperCircuitConfig { ... }

}

}
```

설정 순서가 중요한 이유:

- 회로마다 다른 수의 rotation query를 사용
- Keccak, EVM 같은 복잡한 회로를 먼저 설정
- Sig, ECC Circuit은 halo2-lib의 수직 할당을 사용하므로 마지막에 설정하여 blinding 영역 충돌 방지

Witness 할당 과정

`SuperCircuit` 의 witness 할당은 다음 순서로 진행됩니다:

```
// zkevm-circuits/src/super_circuit.rs

fn  synthesize_sub(&self, config: &Self::Config, ...) -> Result<(), Error> {

// 1. EVM Circuit (3rd phase 필요)

self.evm_circuit.synthesize_sub(&config.evm_circuit, ...)?;

// 2. 나머지 회로들 (2nd phase까지만 필요)

self.keccak_circuit.synthesize_sub(&config.keccak_circuit, ...)?;

self.sha256_circuit.synthesize_sub(&config.sha256_circuit, ...)?;

self.poseidon_circuit.synthesize_sub(&config.poseidon_circuit, ...)?;

self.bytecode_circuit.synthesize_sub(&config.bytecode_circuit, ...)?;

self.tx_circuit.synthesize_sub(&config.tx_circuit, ...)?;

self.sig_circuit.synthesize_sub(&config.sig_circuit, ...)?;
```

```
self.ecc_circuit.synthesize_sub(&config.ecc_circuit, ...)?;

self.modexp_circuit.synthesize_sub(&config.modexp_circuit, ...)?;

self.state_circuit.synthesize_sub(&config.state_circuit, ...)?;

self.copy_circuit.synthesize_sub(&config.copy_circuit, ...)?;

self.exp_circuit.synthesize_sub(&config.exp_circuit, ...)?;

self.pi_circuit.synthesize_sub(&config.pi_circuit, ...)?;

self.rlp_circuit.synthesize_sub(&config.rlp_circuit, ...)?;

self.mpt_circuit.synthesize_sub(&config.mpt_circuit, ...)?;

Ok(())

}
```

데이터 흐름 예시

트랜잭션 하나가 처리되는 전체 과정:

1. **Execution**: Geth가 트랜잭션 실행 → `GethExecTrace` 생성
2. **Bus-Mapping**:
   - `CircuitInputBuilder::handle_tx()` 호출
   - 각 opcode에 대해 `gen_associated_ops()` 실행
   - `ExecStep` 과 `RwOp` 생성
3. **Circuit**:
   - `Block::new_from_block()` 호출
   - 각 회로가 witness 데이터 할당
   - 제약 조건 검증

4. **Proof**:

- Halo2 백엔드로 증명 생성
- 필요시 증명 집계

관련 문서

이전/다음

- **이전**: 개요 - ZKEVM 소개
- **다음**: 크레이트 구조 - 각 Rust 크레이트의 역할

같은 섹션

- 크레이트 구조 - 각 크레이트의 역할과 의존성
- 데이터 흐름 - 상세한 데이터 변환 과정

관련 회로

- EVM Circuit - EVM 실행 검증
- State Circuit - 상태 관리
- Bytecode Circuit - 바이트코드 검증
- Keccak Circuit - 해시 함수
- Tx Circuit - 트랜잭션 검증

관련 시스템

- Bus-Mapping 개요 - 트레이스 변환
- 테이블 시스템 - 회로 간 데이터 공유
- Witness 생성 - 증인 데이터 생성

참고 자료

- `zkevm-circuits/src/lib.rs` : 모든 회로 모듈 정의
- `zkevm-circuits/src/super_circuit.rs` : SuperCircuit 구현
- `bus-mapping/src/circuit_input_builder.rs` : Bus-Mapping 핵심 로직